Opzetten Python voor practicum

Leerdoelen

- ☐ Pythonontwikkelomgeving opzetten in VSCode.
- ☐ Leren werken met virtuele omgevingen in Python.
- ☐ Basis van projectstructuur.

Veiligheid

We gaan programmeren, hier zijn geen veiligheidskwesties aan gebonden. Probeer alleen zoveel mogelijk zelfstandig te programmeren. Het is toegestaan om in dit practicum generatieve AI te gebruiken, gebruik het verstandig:

1. Gebruik het als uitlegmachine, niet als codegenerator
2. Gebruik het om eigen code te laten verbeteren, maar niet herschrijven
3. Gebruik het niet voor volledige oplossingen, snap je iets niet? Vraag het de docent.

Introductie

Python is een van de meest gebruikte programmeertalen binnen de natuurkunde en andere technische vakgebieden. Het wordt gebruikt voor het verwerken en visualiseren van meetgegevens, het uitvoeren van numerieke berekeningen en het aansturen van experimenten. Tijdens de opleiding zullen we Python daarom regelmatig gebruiken, bijvoorbeeld om meetdata te analyseren en grafieken te maken. Python zelf is echter alleen de programmeertaal. Voor praktisch gebruik zijn daarnaast verschillende *packages* nodig. Zo gebruiken we `NumPy` voor numerieke berekeningen, `Matplotlib` voor het maken van grafieken en `SciPy` voor wetenschappelijke berekeningen. Naarmate je meer projecten uitvoert, zul je steeds meer van dergelijke packages tegenkomen.

Hierbij ontstaat een praktisch probleem: niet ieder project gebruikt dezelfde packages of dezelfde versies daarvan. Een nieuwere versie van een package kan bijvoorbeeld anders werken dan een oudere versie. Om te voorkomen dat verschillende projecten elkaar in de weg zitten, werken we daarom met *virtuele omgevingen* (*environments*). Een environment kun je zien als een afzonderlijke Python-installatie met zijn eigen verzameling packages. Voor ieder project kan zo een geschikte en reproduceerbare werkomgeving worden gemaakt.

In dit practicum gebruiken we **Miniforge** om Python, environments en packages te beheren. Miniforge is een relatief compacte Python-distributie die gebruikmaakt van het `conda`-systeem. Miniforge gebruikt standaard packages uit het `conda-forge`-ecosysteem. Hierdoor hoeven packages en hun onderlinge afhankelijkheden niet handmatig geïnstalleerd en bijgehouden te worden.

Voor het daadwerkelijk schrijven en uitvoeren van Python-code gebruiken we **Visual Studio Code (VS Code)**. VS Code is een programmeeromgeving waarin je onder andere Python-scripts en Jupyter notebooks kunt schrijven, uitvoeren en beheren. VS Code bevat daarbij niet zelf “de Python”: we koppelen VS Code aan de Python-interpreter uit de environment die we met Miniforge hebben gemaakt.

In dit practicum ga je daarom stap voor stap een complete Python-werkomgeving opzetten. Je installeert Miniforge en VS Code, maakt vanuit de opdrachtprompt een eigen environment aan, installeert hierin enkele veelgebruikte wetenschappelijke packages en koppelt deze environment vervolgens aan VS Code. Tot slot controleer je met een eenvoudig Python-programma of alles correct is geïnstalleerd.

Voorbereidingsopdracht

1. Neem een laptop mee.
2. Zorg voor een stabiele wifi-verbinding met de HvA.

Python installatie

Opdracht 1 — Oude Anaconda-installatie verwijderen

Voordat we een nieuwe Python-omgeving installeren, verwijderen we eerst eventuele oude installaties van Anaconda. Hiermee voorkomen we dat Windows of VS Code later per ongeluk een oude Python-installatie gebruikt.

1. Open in Windows **Instellingen** → **Apps** → **Geïnstalleerde apps**.
2. Zoek naar **Anaconda** of **Anaconda3**. Indien Anaconda geïnstalleerd is, verwijder deze installatie.
3. Open vervolgens de Verkenner en ga naar je gebruikersmap:
   `C:\Users\<gebruikersnaam>\`
4. Controleer of hier nog een map `anaconda3` aanwezig is. Verwijder deze map indien deze na het verwijderen van Anaconda is achtergebleven.
5. Controleer ook of de volgende verborgen map aanwezig is:
   `C:\Users\<jouw_naam>\anaconda3`
   `C:\Users\<jouw_naam>\miniconda3`
   `C:\Users\<jouw_naam>\.conda`
   `C:\Users\<jouw_naam>\.anaconda`
   `C:\Users\<jouw_naam>\.condarc`
   `C:\ProgramData\Anaconda3`
   Verwijder deze mappen indien deze uitsluitend afkomstig zijn van je oude Anaconda-installatie.
6. Start de computer opnieuw op.

Let op! Heb je eerder zelf environments gemaakt met bestanden die je nog nodig hebt? Verwijder deze dan niet voordat je hebt gecontroleerd of je de bestanden hebt opgeslagen.

Opdracht 2 — Controleren of Anaconda verwijderd is

We controleren nu of Windows niet meer naar de oude Anaconda-installatie verwijst.

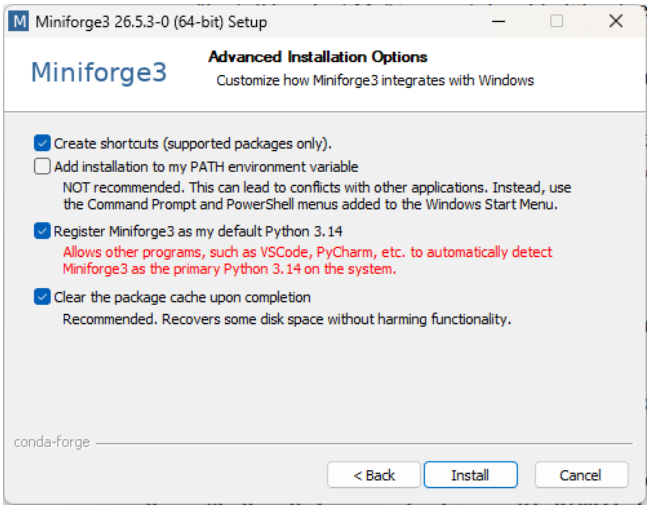
1. Open de Windows **Opdrachtprompt**. Zoek hiervoor in het startmenu naar `cmd` (Command Prompt/Opdrachtenprompt).
2. Voer het volgende commando uit:
   `where conda`
3. Voer vervolgens uit:
   `where python`
4. Controleer de uitvoer. Er mag geen pad meer zichtbaar zijn waarin `Anaconda` of `anaconda3` staat.

Het is geen probleem als Windows meldt dat `conda` niet kan worden gevonden. Op dit moment hebben we immers nog geen nieuwe conda-installatie geïnstalleerd.

Opdracht 3 — Miniforge installeren

We gebruiken **Miniforge** voor het installeren en beheren van Python en de benodigde packages.

1. Ga naar de officiële Miniforge-pagina op GitHub:
   https://conda-forge.org/download/
2. Download de meest recente **Miniforge3**-installer voor jouw besturingssysteem.
3. Kies de installer die past bij je computer. Voor de meeste Windows-computers is dit:
   `Miniforge3-Windows-x86_64.exe`
4. Start het installatiebestand en doorloop de installatie, zie Fig. 1.
5. Gebruik de standaard installatiemap, tenzij je een goede reden hebt om hiervan af te wijken.
6. Rond de installatie af en open daarna vanuit het startmenu de **Miniforge Prompt**.



Figuur 1: Miniforge installatie.

Opdracht 4 — Miniforge controleren

We controleren of Miniforge en het programma `conda` correct zijn geïnstalleerd.

1. Open de **Miniforge Prompt**.
2. Voer het volgende commando uit:
   ```
   conda --version
   ```
3. Als de installatie correct is verlopen, verschijnt een versienummer van conda.
4. Controleer vervolgens welke Python-installatie op dit moment wordt gebruikt:
   ```
   where python
   ```
5. Bekijk het weergegeven pad. Dit pad hoort nu naar je installatie van **Miniforge3** te verwijzen.

Opdracht 5 — Miniforge toevoegen aan PowerShell

Om Conda-commando's rechtstreeks vanuit PowerShell en de terminal van VS Code te kunnen gebruiken, moet Miniforge eenmalig aan PowerShell worden toegevoegd.

1. Open via het Windows-startmenu de **Miniforge Prompt**.
2. Voer het volgende commando uit:
   ```
   conda init powershell
   ```

   Hiermee wordt PowerShell zo ingesteld dat het commando `conda` beschikbaar is.
3. Sluit de Miniforge Prompt en open vervolgens **PowerShell**.
4. PowerShell kan het uitvoeren van het Conda-script blokkeren vanwege de ingestelde beveiliging. Krijg je een melding dat het uitvoeren van scripts is uitgeschakeld (*running scripts is disabled on this system*)? Voer dan in PowerShell het volgende commando uit:
   ```
   Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
   ```

   Bevestig de wijziging wanneer daarom wordt gevraagd door `Y` te typen en op *Enter* te drukken. Door `CurrentUser` te gebruiken is hiervoor geen administratoraccount nodig.
5. Sluit vervolgens alle geopende PowerShell-vensters en **VS Code volledig af**. Start daarna PowerShell opnieuw.
6. Controleer of Conda nu beschikbaar is:
   ```
   conda --version
   ```

Wanneer een versienummer wordt weergegeven, bijvoorbeeld `conda 25.x.x`, is Miniforge correct aan PowerShell toegevoegd.

Opdracht 6 — Een Python-environment maken

We installeren Python niet rechtstreeks als één algemene installatie voor alle projecten. In plaats daarvan maken we binnen Miniforge een afzonderlijke *environment*. Hierdoor kunnen verschillende projecten ieder hun eigen Python-versie en packages gebruiken.
In de vorige opdracht hebben we Conda beschikbaar gemaakt in PowerShell. We gebruiken daarom vanaf nu **PowerShell** voor het uitvoeren van Conda-commando's. De **Miniforge Prompt** kan hiervoor eventueel ook worden gebruikt.

1. Open **PowerShell**.
2. Maak een nieuwe environment met de naam `practicum`:
   ```
   conda create -n practicum python
   ```
3. Bevestig de installatie wanneer daarom wordt gevraagd. Dit doe je door `y` te typen en vervolgens op *Enter* te drukken.
4. Activeer vervolgens de nieuwe environment:
   ```
   conda activate practicum
   ```
5. Controleer de geïnstalleerde Python-versie:
   ```
   python --version
   ```
   Je ziet nu aan het begin van de opdrachtregel `(practicum)` staan. Dit betekent dat de environment `practicum` actief is.
6. Bekijk vervolgens welke environments op je computer aanwezig zijn:
   ```
   conda env list
   ```
   Naast `practicum` zou op dit moment in ieder geval de `base`-environment aanwezig moeten zijn. De actieve environment wordt aangegeven met een sterretje (`*`). De uitvoer ziet er bijvoorbeeld als volgt uit:
   ```
   # conda environments:
   #
   # * -> active
   # + -> frozen
   base                  C:\Users\<jouw_naam>\AppData\Local\miniforge3
   practicum          *   C:
   \Users\<jouw_naam>\AppData\Local\miniforge3\envs\practicum
   ```

Opdracht 7 — Wetenschappelijke packages installeren

Een nieuwe Python-environment bevat nog niet automatisch alle packages die we tijdens de practica nodig hebben. We installeren daarom een aantal veelgebruikte wetenschappelijke packages.
Ga verder in **PowerShell** vanuit de vorige opdracht. De **Miniforge Prompt** kan eventueel ook worden gebruikt.

1. Controleer eerst of de environment `practicum` actief is. Aan het begin van de opdrachtregel moet `(practicum)` staan.
   Is dit niet het geval, activeer de environment dan met:
   ```
   conda activate practicum
   ```

2 Installeer vervolgens NumPy, SciPy, Matplotlib, Pandas en Jupyter (ipykernel):
`conda install numpy scipy matplotlib pandas ipykernel`

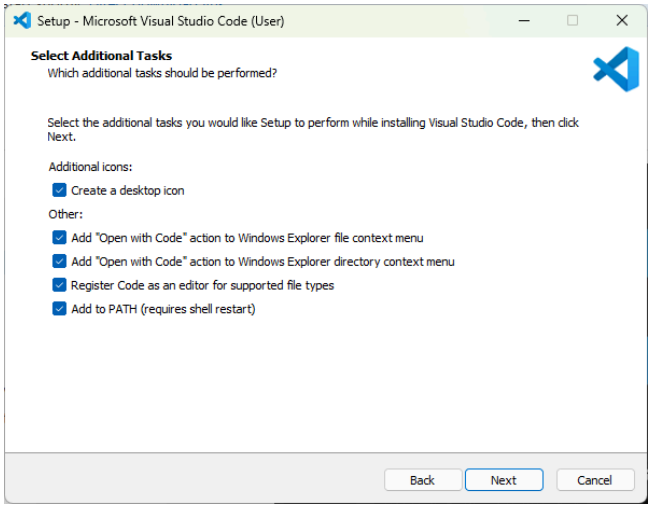
3 Bevestig de installatie wanneer daarom wordt gevraagd door `y` te typen en op *Enter* te drukken.
4 Bekijk na de installatie welke packages in de actieve environment aanwezig zijn:
`conda list`

Vraag: Zoek in de lijst de packages `numpy`, `scipy` en `matplotlib`. Welke versies zijn geïnstalleerd?

Opdracht 8 — Visual Studio Code installeren

Tot nu toe hebben we Python en de benodigde packages geïnstalleerd. We hebben echter nog geen programma waarin we gemakkelijk Python-code kunnen schrijven. Hiervoor gebruiken we **Visual Studio Code (VS Code)**.

1 Ga naar de officiële website van Visual Studio Code: https://code.visualstudio.com/
2 Download en installeer de Windows-versie van VS Code, zie Fig. 2.
3 Start VS Code na de installatie.
4 Open links in VS Code het onderdeel **Extensions**.
5 Zoek naar de extensies **Python, Jupyter, Prettier, Rainbow CSV, Spreadsheet Viewer** en installeer deze.



Figuur 2: VScode installatie.

Mappenstructuur

Mappenstructuur

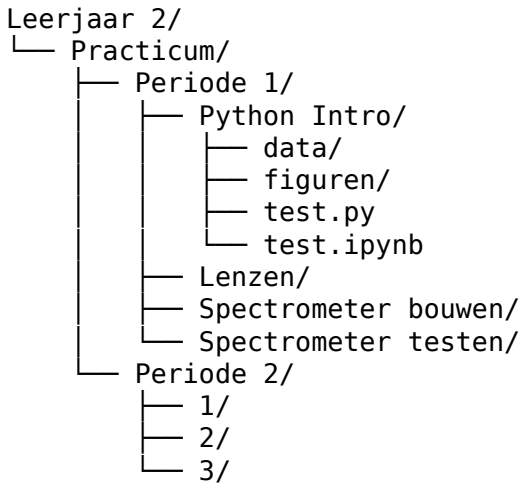
Tijdens practica verzamel je verschillende bestanden, zoals Python-scripts, meetdata en figuren. Het is daarom belangrijk om vanaf het begin een duidelijke **mappenstructuur** te gebruiken. Zo blijven bestanden makkelijk terug te vinden en voorkom je dat verschillende projecten of versies door elkaar raken.
Maak voor ieder practicumserie of project een **aparte hoofdmap** en verdeel deze onder per practicum. Een goede mappenstructuur is een belangrijk onderdeel van zorgvuldig en reproduceerbaar werken.

Opdracht 9 — mappenstructuur

In deze opdrachtg aan we aan de slag om een goede structuur neer te zetten en deze te laden vanuit VScode.

1. Open in VScode **File** → **Open Folder…** → **Documenten**.
2. Creëer een nieuwe map op de locatie die handig is bijvoorbeeld op de **one-drive**.
3. Maak de mappenstructuur na uit Fig. 3.

Ieder practicum dat je start zal je beginnen met **Open Folder…** en dan de juiste practica kiezen. Zo zorg je ervoor dat je altijd je data, figuren en gebruikte scripts kunt terugvinden.



Figuur 3: Boomdiagram voor apparaatcommunicatie.

Test Python

Een **Jupyter Notebook** is een document waarin Python-code in afzonderlijke blokken, zogenaamde *cellen*, kan worden uitgevoerd. Je kunt ze herkennen aan de extensie **.ipynb**[1]. Het grote voordeel hiervan is dat je code stap voor stap kunt uitvoeren en het resultaat direct onder de betreffende cel verschijnt. Naast code kunnen notebooks ook tekst, formules en figuren bevatten.

Dit maakt notebooks bijzonder geschikt voor practica en data-analyse: je kunt meetgegevens inlezen, berekeningen uitvoeren en grafieken maken zonder telkens het volledige programma opnieuw uit te voeren. Tijdens de practica zullen we Jupyter Notebooks daarom regelmatig gebruiken.

Opdracht 10 — Test Jupyter Notebook (.ipynb)

In de vorige opdrachten hebben we de basis gelegd voor een overzichtelijke mappenstructuur en hebben we de benodigde software geïnstalleerd. Nu gaan we controleren of alles correct werkt.

1. Open in VS Code via **File** → **Open Folder…** de map **Practicum Python Intro**.
2. Open het bestand `script.ipynb` en voeg met `+ Code` een nieuwe codecel toe.
3. Neem de onderstaande code over en voer de cel uit met `Shift+Enter`.
4. Als er nog geen Python-omgeving is geselecteerd, vraagt VS Code om een *kernel* te kiezen. Selecteer hier de eerder aangemaakte `practicum_omgeving`. Deze kan soms wat verstopt zitten.
5. Je kunt de actieve omgeving ook controleren of wijzigen via `Select Kernel` rechtsboven in het notebook. Controleer of hier `practicum_omgeving` is geselecteerd.
6. Controleer of de uitvoer van de code overeenkomt met wat je verwacht.

script.ipynb

```python
print("Hello world")

import numpy as np

a = np.array([1, 2, 3, 4])

print("Array a:", a)
print("Array a^2:", a**2)
```

Naast Jupyter Notebooks wordt Python veel gebruikt in de vorm van **Python-scripts**. Dit zijn tekstbestanden met de extensie **.py**. In tegenstelling tot een Jupyter Notebook bestaat een Python-script niet uit losse cellen: bij het uitvoeren wordt het programma in principe van boven naar beneden uitgevoerd.

Bij een Jupyter Notebook hebben we hiervoor een *kernel* geselecteerd. Voor een Python-script moet VS Code weten welke **Python-interpreter** gebruikt moet worden. Het is belangrijk om hier onze eerder gemaakte `practicum_omgeving` te kiezen. Anders kan het gebeuren dat het script met een andere Python-installatie wordt uitgevoerd waarin bijvoorbeeld onze packages niet zijn geïnstalleerd.

Opdracht 11 — Test Python-script (.py)

1. Zorg dat de map **Practicum Python Intro** nog geopend is in VS Code.
2. Open het bestand met de naam `script.py`.
3. Selecteer de juiste Python-interpreter. Druk hiervoor op `Ctrl+Shift+P` en zoek naar: `Python: Select Interpreter`

[1] Zorg ervoor dat extensies zichtbaar zijn bij bestanden. Kun je dit niet zien, spreek dan even de practicumbegeleider aan.

4. Selecteer de eerder gemaakte `practicum_omgeving`. Als deze niet direct zichtbaar is, kijk dan bij de overige beschikbare Python-omgevingen.
5. Neem onderstaande code over in `script.py` en sla het bestand op.
6. Voer het programma uit met de knop **Run Python File** rechtsboven in VS Code.
7. Onderaan VS Code wordt nu een **Terminal** geopend. Controleer of het programma zonder foutmeldingen wordt uitgevoerd.

script.py

```python
import sys
import numpy as np

print("Hello world")

a = np.array([1, 2, 3, 4])

print("Array a:", a)
print("Array a^2:", a**2)

print("\nPython wordt uitgevoerd vanuit:")
print(sys.executable)
```

Opdracht 12 — Controleer de Python-omgeving

Bekijk de laatste regel van de uitvoer van je programma. Hier staat het pad naar het bestand `python.exe` waarmee het programma is uitgevoerd.

1. Controleer of in dit pad de naam `practicum_omgeving` voorkomt.
2. Controleer ook linksonder in VS Code welke Python-versie en environment geselecteerd zijn.
3. Staat hier niet `practicum_omgeving`? Selecteer dan opnieuw de juiste interpreter via `Python: Select Interpreter` en voer `script.py` opnieuw uit.

Afronding

1. Heb je alle opdrachten afgerond? Laat dit dan controleren door de practicumbegeleider.
2. Lukt het niet om alles op tijd af te ronden, zorg ervoor dat je dit dan thuis afrond! Mocht je er thuis niet uitkomen, spreek dan de practicumbegeleider even aan, diegene zal dan met je meekijken.

Appendix: Materialen Lijst Practica

- Laptop met stabiele internetverbinding.